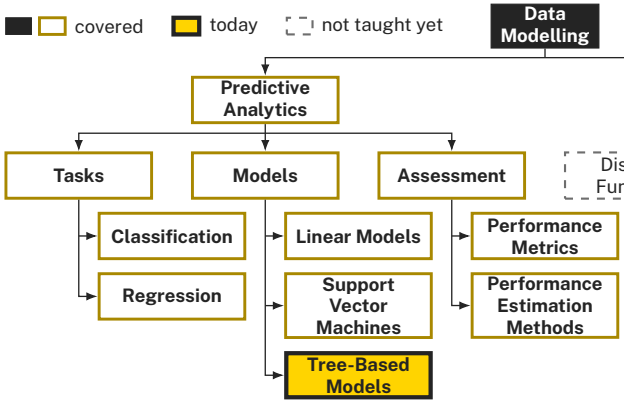
DALHOUSIE UNIVERSITY

Tree-Based Models

Recursive partitioning, regression trees, and pruning

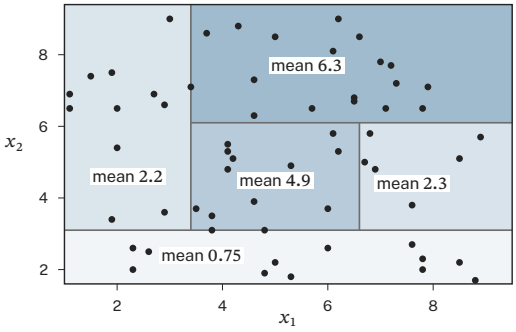
Prof. Gabriel Spadon, PhD
Faculty of Computer Science
spadon@dal.ca

Where Tree-Based Models Sit



Today, tree-based models. Regression trees, and pruning back to the size the evidence supports.

Sixty Cases From Five Regions



Region	mean	cases
$x_2 < 3.1$	0.75	12
$x_2 \geq 3.1,\ x_1 < 3.4$	2.2	11
$3.4 \leq x_1 < 6.6,\ 3.1 \leq x_2 < 6.1$	4.9	13
$x_1 \geq 6.6,\ 3.1 \leq x_2 < 6.1$	2.3	6
$x_1 \geq 3.4,\ x_2 \geq 6.1$	6.3	18

Two numeric predictors, x_1 and x_2, and a numeric target y, the region's mean plus noise of standard deviation 0.6, drawn with a seed.

We drew the data, so the right answer is known, five regions.

Regression Trees

Same algorithm, new target. A leaf predicts a mean, and a question is scored by squared error.

Classification Tree to Regression Tree

	Classification tree	Regression tree
Target	nominal	numeric
A leaf predicts	the majority class of its cases (and the class distribution)	the mean of its cases' target values
A split is scored by	impurity reduction: Gini or information gain	error reduction: the drop in within-node sum of squares
In `rpart`	`method = "class"`	`method = "anova"`
In scikit-learn	`DecisionTreeClassifier`	`DecisionTreeRegressor`

The algorithm is unchanged. List every question, score it, keep the best, split, repeat. Only two things are swapped, what a leaf predicts and how mixed a box is.

Why a Leaf Predicts the Mean

One leaf of the fitted tree holds six cases.

$$1.95,\ 3.22,\ 2.29,\ 1.16,\ 2.69,\ 2.28$$

The leaf must predict one number k for all six. Score a guess by its **mean squared error**, the average of $(y - k)^2$.

guess k	mean of $(y - k)^2$
2	0.473
2.5	0.458
mean, 2.265	**0.402**

No other number does better. Moving k off the mean adds more squared error on the far side than it removes on the near side.

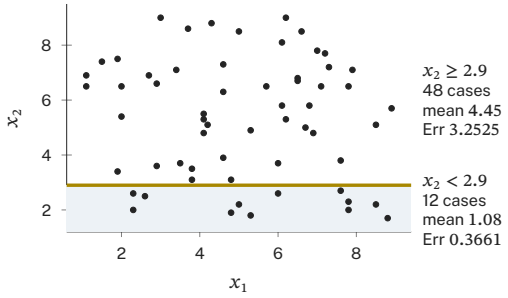
At the mean the squared distances are 0.099, 0.912, 0.001, 1.221, 0.181 and 0.000. They add to 2.414, the leaf's **SSE**, its sum of squared errors. Divided by the 6 cases it is the leaf's **Err**, $2.414/6 = 0.402$, its variance with n in place of $n - 1$. Err plays the part Gini played, and it is 0 when every case equals the mean.

R prints the leaf as one line, its node, answer, size, SSE and mean.

```
28) x1>=6.45 6 2.41 2.26 *
```

scikit-learn stores Err itself, `tree_.impurity`, 0.402.

Scoring the First Question



All 60 cases have Err $= 4.4986$. Ask $x_2 < 2.9$. The **pooled error** weights each side's Err by its share of the cases.

$$\frac{12}{60}(0.3661) + \frac{48}{60}(3.2525) = 2.6752$$

The question removes $4.4986 - 2.6752 = \mathbf{1.823}$, the most of all 42 cuts on x_2 and 38 on x_1.

In sums of squares, parent minus both sides, the same drop is $269.915 - 4.393 - 156.118 = 109.404$, which is 60×1.8234. Pruning prices questions in these units.

The Search as Code

DALHOUSIE UNIVERSITY

R

```r
err <- function(v) mean((v - mean(v))^2) # Err(D)
gains <- function(x, y) {   # error removed per cut
  v <- sort(unique(x))
  cuts <- (head(v, -1) + tail(v, -1)) / 2
  g <- sapply(cuts, function(c) {
    l <- y[x < c]; r <- y[x >= c]
    err(y) - (length(l) * err(l) +
              length(r) * err(r)) / length(y)
  })
  list(cuts = cuts, g = g)
}
round(err(d$y), 4)                    # 4.4986
for (col in c("x1", "x2")) {  # best cut per predictor
  s <- gains(d[[col]], d$y)
  cat(col, length(s$cuts), s$cuts[which.max(s$g)],
      round(max(s$g), 4), "\n")
}
# x1 38 3.2 0.9684
# x2 42 2.9 1.8234
```

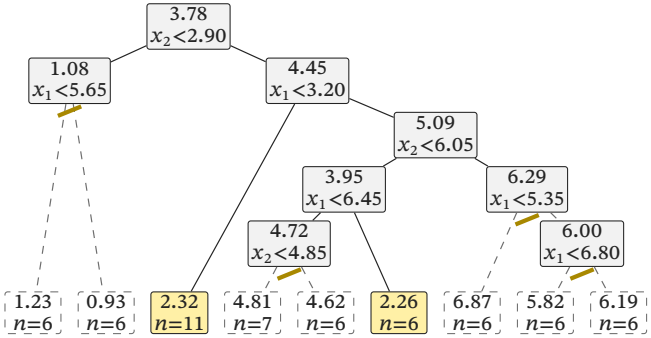
Python

```python
def err(v):                           # Err(D)
    return np.mean((v - v.mean()) ** 2)

def gains(x, y):          # error removed by each cut
    v = np.unique(x)
    cuts = (v[:-1] + v[1:]) / 2     # midpoints
    g = [err(y) - (len(y[x < c]) * err(y[x < c])
         + len(y[x >= c]) * err(y[x >= c])) / len(y)
         for c in cuts]
    return cuts, np.array(g)
round(err(y), 4)                      # 4.4986
for col in ["x1", "x2"]:   # best cut per predictor
    cuts, g = gains(X[col].to_numpy(), y.to_numpy())
    print(col, len(cuts), round(cuts[g.argmax()], 2),
          round(g.max(), 4))
# x1 38 3.2 0.9684
# x2 42 2.9 1.8234
```

`gains` is the Gini search with `err` in its place. Each output line, predictor, cuts tried, best cut, drop.

Recursion Gives Nine Leaves



dashed = removed by the 1-SE rule
gold = a leaf after pruning; a slash marks a collapsed node

The same search runs inside each side, again and again, stopping only where a split would leave a leaf under 6 cases.

- Each box prints the mean of its cases and its question.
- 9 leaves for 5 true regions. The 4 dashed splits divide cases that come from one region.
- Their SSE adds to 15.95, down from the root's 269.92.

A Step Function, Never a Slope

Two new cases, $(x_1, x_2) = (5, 7)$ and $(9.5, 9.5)$.

model	$(5, 7)$	$(9.5, 9.5)$
9-leaf tree	6.87	6.19
5-leaf tree	6.29	6.29
straight line	4.88	7.33

- A tree's prediction is the mean of one leaf, so it is constant inside each rectangle and jumps at each cut.
- It can never predict above the largest training y, 7.37, however far out the case lies. The line extrapolates, 7.33 and climbing.
- Inside the data, the steps fit the regions. Ten-fold MSE, 0.80 for the 5-leaf tree and 2.67 for linear regression.

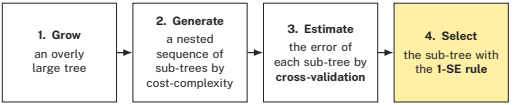
Pruning

Grow large, then keep the smallest subtree whose cross-validated error is close to the best.

What a Stopping Rule Cannot See

A stopping rule is fixed before the tree grows, so it judges every split by the same yardstick.

- With no rule the tree grows 60 leaves of one case each, training SSE 0.
- A smallest leaf of 6 gives 9 leaves. Four splits remove 24.8 to 109.4 in SSE. Four remove 3.02 or less, all inside one region.
- `rpart`'s default smallest leaf, 7, gives 4 leaves. The region with mean 2.3 holds only 6 cases, so a real split is never made.



Stopping early is also called pre-pruning. Post-pruning grows a large tree first, then lets cross-validation decide which branches to remove.

Charging a Price per Leaf

$$R_\alpha(T) = R(T) + \alpha \times \text{leaves}$$

$R(T)$ is a subtree's training SSE. α, the **complexity parameter**, is the price of one leaf in units of SSE, and $R_\alpha(T)$ is the subtree's **cost-complexity**. A leaf is worth keeping only if it lowers $R(T)$ by more than α.

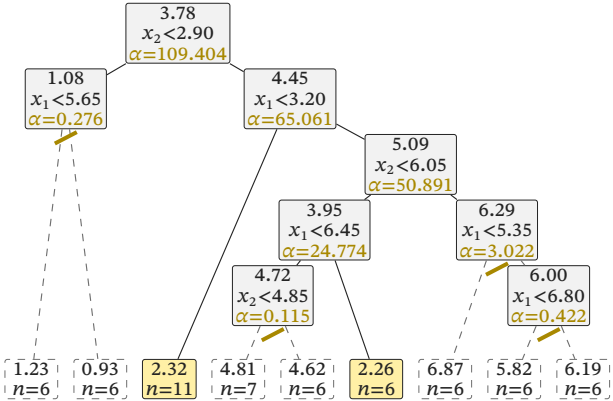
At $\alpha = 5$ the 5-leaf tree costs $19.79 + 5 \times 5 = 44.79$, the 6-leaf tree $16.76 + 30 = 46.76$, the 4-leaf tree $44.56 + 20 = 64.56$. Five leaves win.

$\alpha = 0$ keeps all 9 leaves, $\alpha = 1$ keeps 6, and a large α keeps only the root.

leaves	$R(T)$	cost, $\alpha = 1$	cost, $\alpha = 5$
9	15.95	24.95	60.95
8	16.07	24.07	56.07
7	16.34	23.34	51.34
6	16.76	**22.76**	46.76
5	19.79	24.79	**44.79**
4	44.56	48.56	64.56

Each row is the lowest-SSE subtree of that size. A higher price favours a smaller tree.

The Weakest Link Goes First



A question's price is what its branch saves per extra leaf, its SSE as one leaf minus its leaves' SSE, over leaves -1.

- $x_1 < 5.35$, 18 cases, SSE 6.85 as one leaf and 3.41 in 3, $(6.85 - 3.41)/2 = 1.72$.
- Inside it, $x_1 < 6.80$ saves $2.37 - 1.95 = 0.42$ and goes first.
- Then $x_1 < 5.35$ has 2 leaves, $6.85 - 3.83 = 3.02$.

The weakest link, $x_2 < 4.85$ at 0.115, goes first of all. Each α is a question's price when it goes.

The Pruning Sequence

Leaves	$R(T)$, SSE	α, its price	10-fold MSE	SE
9	15.95	0.000	0.753	0.381
8	16.07	0.115	0.753	0.381
7	16.34	0.276	0.753	0.381
6	16.76	0.422	0.753	0.381
5	**19.79**	**3.022**	**0.803**	0.387
4	44.56	24.774	1.209	0.528
3	95.45	50.891	2.063	0.705
2	160.51	65.061	2.955	0.668
1	269.92	109.404	4.396	0.734

Raise α from 0 and the questions go in the order of their prices. Each row is one subtree, nested in the one above, and a branch once removed never returns.

Read a row as *a smaller tree for a small rise in training error*. From 6 leaves to 5 costs 3.02 in SSE. From 5 to 4 costs 24.77, eight times more, because that split separates two real regions.

Scoring One Subtree on Ten Folds

Take the 5-leaf row. Each fold regrows the tree on 54 cases, prunes it at the same α and predicts the 6 it left out.

fold	1	2	3	4	5
MSE	0.623	**4.236**	0.225	0.260	0.351
fold	6	7	8	9	10
MSE	0.573	0.217	0.788	0.244	0.511

- The mean, $8.028/10 = \mathbf{0.803}$, is the cross-validated MSE.
- The ten scores have standard deviation $s = 1.222$, so the **standard error** is $s/\sqrt{10} = \mathbf{0.387}$.
- Fold 2 holds $(4.8, 3.1)$ and $(3.8, 3.5)$, just above the low region. Without them the fold's tree cuts at $x_2 < 3.65$ and predicts 1.90 for both.

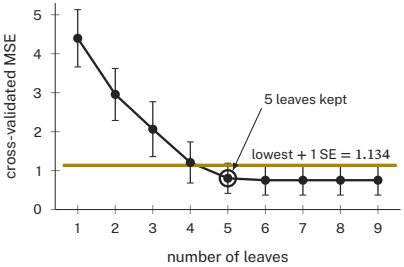
R `fold_mse(tree5)` and Python `cross_val_score(DTR(ccp_alpha=a[4], **kw), X, y, cv=f10)` print the same ten scores. That one bad fold is most of this SE.

Choosing the Size by Cross-Validation

Training SSE always prefers 9 leaves. Choose on the cross-validated MSE, the way K was chosen for kNN.

- The lowest MSE, 0.753, is shared by 6 to 9 leaves, with SE 0.381.
- The **one-standard-error rule** keeps the smallest tree at or below $0.753 + 0.381 = 1.134$.
- 5 leaves score 0.803, inside. 4 score 1.209, outside. Keep **5 leaves**.

Which of the flat 6 to 9 leaves scores lowest is up to the folds, as the depth search drifted to depth 10 on a flat curve. A gap under one SE is noise.



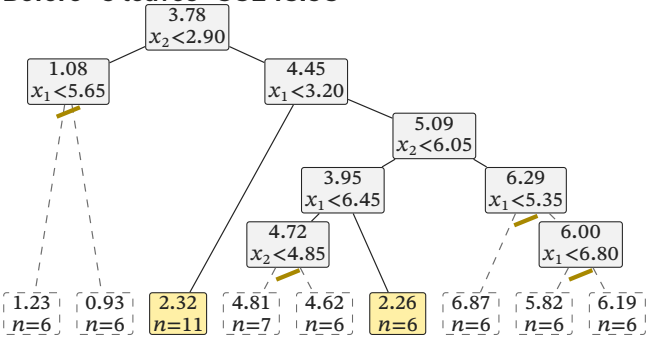
The x-**SE rule** draws the line at lowest + x SE.

x	0	0.5	1	2
leaves, Python	6	5	5	4
leaves, R	7	6	5	4

Before and After Pruning

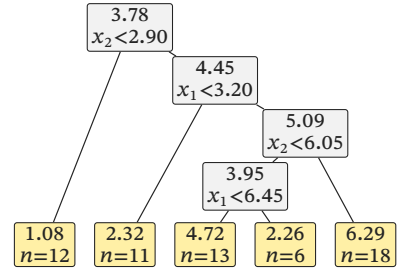
DALHOUSIE UNIVERSITY

Before · 9 leaves · SSE 15.95



dashed = removed by the 1-SE rule
gold = a leaf after pruning; a slash marks a collapsed node

After · 5 leaves · SSE 19.79



Training SSE rose 24%. The five leaves hold 12, 11, 13, 6 and 18 cases, and every case in a leaf comes from the same true region.

Pruning in R and Python

DALHOUSIE UNIVERSITY

R

```r
library(rpart); library(rpart.plot)
fold <- rep(1:10, each = 6)          # 10 blocks of 6
fit <- rpart(y ~ x1 + x2, data = d,
  method = "anova",   # cp = 0 grows large
  control = rpart.control(cp = 0, minbucket = 6,
    minsplit = 12, xval = fold))
printcp(fit)   # CP nsplit rel error xerror xstd
tab <- fit$cptable                   # 1-SE rule
best <- which.min(tab[, "xerror"])
lim <- tab[best, "xerror"] + tab[best, "xstd"]
row <- min(which(tab[, "xerror"] <= lim))
pruned <- prune(fit, cp = tab[row, "CP"])
rpart.plot(pruned)                   # 5 leaves
```

Python

```python
kw = {"min_samples_leaf": 6, "random_state": 3141}
base = DTR(**kw).fit(X, y)           # grow large
path = base.cost_complexity_pruning_path(X, y)
a = path.ccp_alphas                  # one per subtree
f10 = KFold(10)                      # 10 blocks of 6
e, se = [], []                       # CV error, its SE
for c in a:
    s = -cross_val_score(DTR(ccp_alpha=c, **kw),
        X, y, cv=f10,
        scoring="neg_mean_squared_error")
    e.append(s.mean())
    se.append(s.std(ddof=1) / np.sqrt(10))
e, se = np.array(e), np.array(se)
ok = e <= e.min() + se[e.argmin()]  # 1-SE rule
pruned = DTR(ccp_alpha=a[ok].max(), **kw).fit(X, y)
```

`xval = fold` gives `rpart` the ten folds of `KFold(10)`. scikit-learn returns the α sequence and training impurities but no error estimate, so the loop scores each subtree. A classification tree prunes the same way. On `Glass` both grow 50 leaves and keep 8.

The cp Table R Prints

CP	nsplit	rel error	xerror	xstd
0.40533	0	1.0000	1.0320	0.1093
0.24104	1	0.5947	0.7153	0.1031
0.18854	2	0.3536	0.5021	0.1070
0.09178	3	0.1651	0.2766	0.0821
0.01120	4	0.0733	0.1992	0.0766
0.00156	5	0.0621	0.1737	0.0600
0.00102	6	0.0605	0.1674	0.0600
0.00043	7	0.0595	0.1674	0.0600
0.00000	8	0.0591	0.1674	0.0600

Every column is over the root's SSE, 269.92, or its Err, 4.4986, so the table has no units.

- One row per subtree, with `nsplit` + 1 leaves.
- `rel error` is $R(T)/269.92$. At 5 leaves, $19.79/269.92 = 0.0733$.
- `CP` is $\alpha/269.92$. At 5 leaves, $3.022/269.92 = 0.0112$.
- `xerror` is the cross-validated MSE over 4.4986, and $0.1674 \times 4.4986 = 0.753$. `xstd` is its SE, taken from the 60 cases' squared errors.

The 1-SE line is $0.1674 + 0.0600 = 0.2274$. The first row under it has `nsplit` 4, **5 leaves**.

What a Regression Tree Costs

	Linear regression	Pruned regression tree
Shape it can fit	one plane	steps, any number of regions
Ten-fold MSE here	2.67	0.80, or 1.15 with α chosen inside each fold
Beyond the data	extrapolates	flat at the last leaf's mean
Reads as	one weight per predictor	five rules
Setting to tune	none	α, by cross-validation

Use a tree when the target jumps between regions, when predictors interact, or when the model must read as rules. Prefer a line when the target changes smoothly, or when you must predict outside the range you trained on.

The 0.80 reuses the folds that chose α, so it flatters the tree. Choosing α inside each outer fold, as the depth was chosen, gives 1.15 in Python and 1.17 in R, still far below the line.

Three Ideas to Keep

DALHOUSIE UNIVERSITY

Same algorithm

A leaf predicts its mean. A question is worth the SSE it removes, parent minus both sides.

A price per leaf

Raising α removes the cheapest question first. Each size gets one nested subtree.

Choose on folds

Cross-validate every subtree and keep the smallest within one SE of the best.